Robust Malicious Network Traffic Detection Framework With Automated Drift Detection, Identification, and Adaptation

Xueying Han, *Member, IEEE*, Jian Qin, *Student Member, IEEE*, Changzhi Zhao, Weike Fang, Junrong Liu, Weihang Wang, Bo Jiang, Susu Cui, Zhigang Lu, and Baoxu Liu

***Abstract*—The rise in network attacks has made robust malicious traffic detection crucial. However, the dynamic nature of network traffic causes concept drift, undermining the efficacy of traditional detection methods, which often rely on a static i.i.d data environment and struggle to adapt to new patterns. To overcome these limitations, we propose Argus, a novel framework for malicious traffic detection that operates in a comprehensive, automated, and adaptive manner. Argus tackles three core challenges: accurately classifying known traffic while detecting drift, automatically identifying malicious drifting traffic, and maintaining performance through continuous updates. To address these challenges, Argus integrates a contrastive learning-based module to produce compact representations of traffic and implements a fine-grained drift detection method using category-specific reconstruction loss distributions. For drifting traffic, Argus uses clustering-based automated identification to detect attacks without human intervention. Furthermore, a distance-constrained update mechanism ensures smooth model adaptation, preserving stability and accuracy. Extensive experiments demonstrate that Argus achieves superior performance, with an average F1 score exceeding 95% under various conditions and retaining robust performance even under extreme drift scenarios.**

***Index Terms*—Malicious traffic detection, concept drift, contrastive learning, drift identification, model adaptation.**

I. Introduction

In recent years, network attacks have become increasingly prevalent, making robust malicious traffic detection a critical component of maintaining network security [1], [2]. However, the growing complexity and dynamism of network traffic have introduced the challenge of *concept drift*, posing significant difficulties for robust detection [3]. Specifically, existing malicious traffic detection methods [4], [5], [6] are based on the assumption that training and testing data are independent and identically distributed (i.i.d.). However, in open-world environments, changes in legitimate users' behavior patterns [7] and the advancement of attackers' techniques [8] often cause traffic distributions to diverge from the original data. This phenomenon, known as concept drift, undermines the effectiveness of these methods and increases false positive and negative rates.

Several solutions have been proposed to address the issues caused by concept drift. However, they fall short of providing a *comprehensive*, *automated*, and *adaptive* approach for malicious traffic detection in the presence of drifting traffic. Existing solutions can be divided into three approaches: (1) One approach is to periodically update the model by incorporating new labeled data to enhance the model's ability to adapt to novel traffic [9], [10]. However, this method requires a large amount of labeled drifting data, which is difficult and costly to obtain. Moreover, concept drift at test time is inherently relative to the training data, meaning that retraining the model cannot fully address the underlying issue. (2) Another approach focuses on enhancing model robustness through feature augmentation [2], [11], [12], [13] and pretraining [14], [15]. Their goal is to make the model insensitive to variations or perturbations. However, they can only handle minor drifts and cannot manage significant drifts resulting from the emergence of new attack techniques. (3) Drift detection has also emerged as a promising strategy [16], [17], [18], [19], which identifies changes in traffic patterns by evaluating confidence scores or sample distances. However, these methods can only detect the presence of drift. Determining the characteristics of drift traffic, specifically whether it is generated by malicious activities, still requires manual intervention. When drift alerts occur frequently, this places a heavy burden on operational staff, leading to fatigue and increasing the risk of overlooking real threats [20]. Furthermore, although some methods provide adaptation to drifting traffic, they still require manual selection and labeling of samples for adaptation [19], [21].

To provide a comprehensive, automated, and adaptive solution addressing the limitations of existing methods, this paper proposes a robust network malicious traffic detection

Received 7 May 2025; revised 22 December 2025, 16 March 2026, and 26 April 2026; accepted 9 May 2026. Date of publication 18 May 2026; date of current version 26 May 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFC2206402; in part by the Youth Innovation Promotion Association Chinese Academy of Sciences (CAS) under Grant 2021156; in part by the Program of Key Laboratory of Network Assessment Technology, CAS; and in part by the Program of Beijing Key Laboratory of Network Security and Protection Technology. The associate editor coordinating the review of this article and approving it for publication was Dr. Simone Soderi. *(Corresponding authors: Bo Jiang; Susu Cui.)*

Xueying Han, Jian Qin, Changzhi Zhao, Junrong Liu, Bo Jiang, Susu Cui, Zhigang Lu, and Baoxu Liu are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: hanxueying@iie.ac.cn; qinjian@iie.ac.cn; zhaochangzhi@iie.ac.cn; liujunrong@iie.ac.cn; jiangbo@iie.ac.cn; cuisusu@iie.ac.cn; luzhigang@iie.ac.cn; liubaoxu@iie.ac.cn).

Weike Fang and Weihang Wang are with the University of Southern California, Los Angeles, CA 90089 USA (e-mail: weikefan@usc.edu; weihangw@usc.edu).

Digital Object Identifier 10.1109/TIFS.2026.3694664

framework named **Argus**. Argus focuses on three challenges: (1) How to achieve accurate classification of known i.i.d. traffic while simultaneously detecting drifting traffic? (2) How to automatically identify the characteristics of drifting traffic without human intervention? (3) How to maintain optimal performance while continually adapting to drifting traffic, enabling further enhancement over time?

For the first challenge, we design a novel module named ACLearner based on contrastive learning and autoencoders to generate compact representations for each type of traffic. ACLearner is trained separately on both statistical features and behavioral features, and they are integrated in a bagging manner to cover a wider range of categories, helping prevent single points of failure and improving the generalization capability. Additionally, we introduce a novel fine-grained measurement method to detect traffic drift and classify non-drifting traffic. It evaluates the reconstruction loss distribution of the nearest class in the latent feature space, enabling it to more precisely handle the distributional differences among various traffic categories, thereby improving the discriminative ability of the detection method.

To address the second challenge, for traffic identified as drift, we design an automated drift identification strategy based on clustering distribution differences, eliminating the need for human intervention. This strategy detects malicious traffic by analyzing its clustering characteristics in both the feature and physical spaces. Our analysis shows that, due to the certainty of its attack purpose and target, malicious traffic typically exhibits stronger clustering while normal traffic tends to be more dispersed.

Regarding the third challenge, we introduce an adaptation method that can use non-drifting or drifting traffic to update Argus as needed. To prevent significant degradation in the detection performance on old data during model updates, we design a distance-constrained update method that imposes restrictions on changes in the feature space throughout the update process. This ensures that the model or detection criteria do not undergo abrupt changes while aligning the update process with the architecture of Argus.

We implement Argus and conduct thorough experiments to assess its effectiveness in handling various types and degrees of drifting traffic. The experimental results demonstrate that Argus outperforms other methods across different scenarios, with an average F1 exceeding 95%, and maintains stable performance under varying degrees of drift. Even under extreme drift conditions, its F1 score remains at 88.22%, significantly surpassing other methods, which generally experience a drop in F1 by more than 40%. Additionally, we validate the effectiveness of individual components of Argus through ablation studies. For instance, the drift identification module accurately classifies drifting traffic as malicious or normal without human intervention, consistently achieving an accuracy of over 97% and greatly outperforming versions without this component.

In summary, this paper makes the following contributions:

- We propose Argus,[1] a novel framework that robustly detects malicious network traffic in complex environments through automated classification, drift detection and identification, and model adaptation. To our knowledge, this is the *first* comprehensive, automated and adaptive detection framework designed to handle malicious network traffic with concept drift.
- We design ACLearner to generate compact representations for different types of traffic, along with a fine-grained method to detect drift and classify non-drifting traffic. We implement an automated mechanism based on clustering characteristics to identify drifting traffic types. To maintain optimal performance, we also introduce a distance-constrained update strategy for the proposed model.
- We conduct extensive experiments to demonstrate the effectiveness of Argus, and results show that Argus outperforms other methods across different scenarios and maintains stable performance even under extreme drift conditions. The functionality of each component of Argus has also been thoroughly validated.

[1] https://github.com/Argusaaa/Argus

II. Preliminary

A. System Model and Scope

To ensure comprehensive monitoring coverage, the proposed framework is deployed at the mirroring port of the network's core switch. This strategic placement provides visibility into both North-South traffic (external ingress/egress) and East-West traffic (internal communications), thereby effectively facilitating the detection of both external intrusions and internal threats. Our detection scope focuses on network attacks that manifest discernible patterns within traffic flows, including but not limited to botnet attacks, DoS attacks and web attacks. Regarding data availability, we assume access to a labeled dataset containing both normal and malicious traffic for the initial offline training phase. However, during the online deployment phase, the system must process continuous streams of unlabeled traffic and adapt itself to concept drift without requiring real-time ground-truth labels from human analysts.

B. Threat Model

We consider an open-world scenario where an active external adversary operates under a black-box assumption, aiming to evade detection without knowledge of the model's parameters or training data. In this setting, concept drift is driven by two forces: benign environmental evolution and malicious adversarial dynamics. For normal traffic, drift stems from natural shifts in the network environment such as evolving business demands, infrastructure changes, or shifting user behaviors. For malicious traffic, adversaries induce drift through either attack evolution, which involves upgrading tools and modifying signatures to evade static rules, or novel attacks, where 0-day vulnerabilities or new methods generate patterns unseen during training.

Argus is most effective against coordinated attacks with discernible concentrations, such as high-volume similar traffic or tight physical connectivity. However, highly dispersed activities, including low-frequency scans or distributed attacks

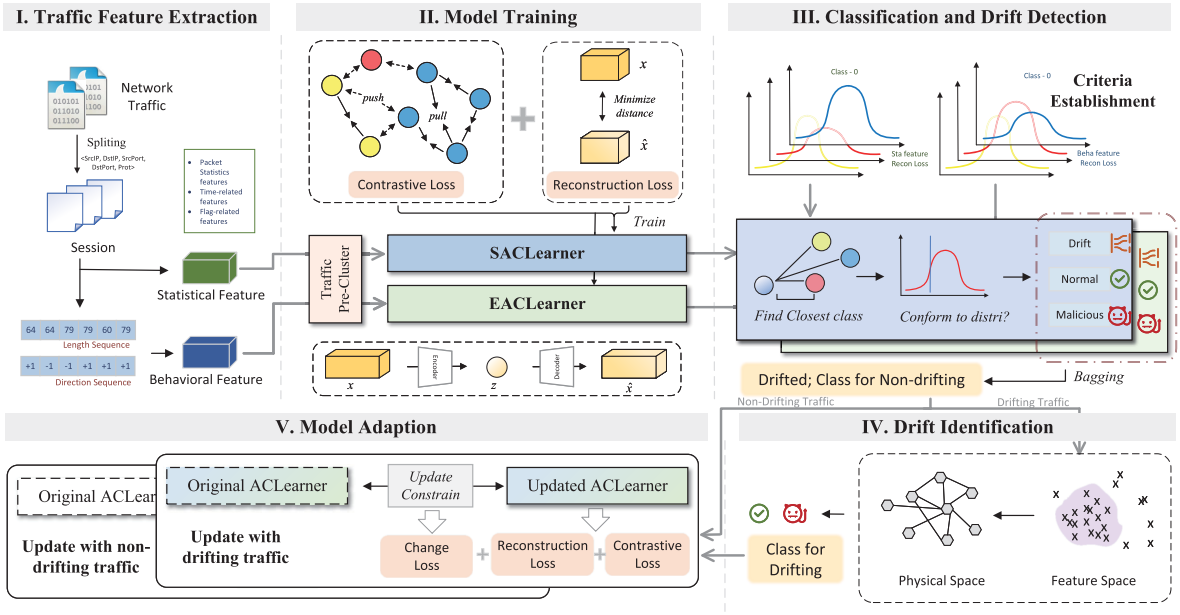

Fig. 1. Workflow of argus.

on unrelated hosts, remain challenging to detect because they mimic the natural sparsity of benign users and represent the inherent boundary of our distribution-based approach.

C. Problem Definition

Let $\mathcal{X}$ be the feature space and $\mathcal{Y}$ be the label space. A network malicious traffic detection model aims to learn a joint probability distribution $P(x, y) = P(x)P(y|x)$ from a labeled training set, where $x \in \mathcal{X}$ represents a traffic sample and $y \in \mathcal{Y}$ denotes its label. In a static environment, the model can accurately classify data based on this learned distribution. However, in real-world dynamic networks, the traffic distribution is time-variant. Concept drift occurs when the joint distribution at time t differs from that during training, i.e., $P_t(x, y) \neq P_{train}(x, y)$. This deviation manifests in two primary forms: deviations in the marginal probability distribution of input features, denoted as $P(x)$ drift, which typically corresponds to the evolution of normal user behaviors; and deviations in the conditional probability distribution or the expansion of the label space $\mathcal{Y}$, denoted as $P(y|x)$ drift, which involves changes such as the emergence of new attack classes. Such drifts often lead to model biases, resulting in increased false positives or false negatives.

Our objective is to develop an efficient, robust, and fully automated detection system, Argus, capable of maintaining superior performance under these drift conditions. Specifically, Argus is designed to achieve three core objectives: (1) to identify normal traffic and known malicious traffic that aligns with the learned distribution $P(x, y)$ with high accuracy; (2) to promptly detect samples that deviate from $P(x)$ or $P(y|x)$ and automatically categorize this drifting traffic as either benign or malicious without the human intervention; (3) to be adaptive, capable of updating its learned probability distribution to accommodate dynamic changes.

III. Design of Argus

A. Workflow of Argus

Argus is a comprehensive robust malicious network traffic detection framework that can effectively handle dynamic traffic. It consists of five key components, including traffic extraction module (I, Sec III-B), model training module (II, Sec III-C), classification and drift detection module (III, Sec III-D), drift identification module (IV, Sec III-E) and model adaptation module (V, Sec III-F). Figure 1 illustrates an overview of Argus.

During training, statistical features and behavioral features of traffic are first extracted by I, and II uses these features to train two separate ACLearners, both based on contrastive learning and autoencoders. During inference, traffic is also first processed by I, followed by the extraction of latent features and reconstruction loss using the ACLearners trained by II. III then determines whether the traffic exhibits drift and classifies its category by comparing it with the criteria. The detection criteria in III are based on the models trained in II and the data used during training. For non-drifting traffic, III directly outputs its category. For drifting traffic, IV is responsible for determining whether they are malicious according to cluster characteristics. To maintain stable model performance, V can use both drifting traffic from IV and non-drifting traffic from III for updates.

B. Traffic Feature Extraction

Network traffic can be categorized into data transfer traffic and management and control traffic. The former typically has a longer duration, larger data volume, and a greater number of packets, while the latter is characterized by a shorter duration and fewer packets. Given the diversity of network traffic mentioned earlier, we extract both statistical features

and behavioral features of network traffic to better characterize their attributes.

We use the session as the processing unit, where a session is defined by packets sharing the same 5-tuple (Src IP, Dst IP, Src Port, Dst Port, and Protocol). To ensure feature completeness, a session is passed to Argus for feature extraction only after its transmission is fully completed. The statistical features we extract from sessions include packet statistics such as average packet length and average packet size in bytes, time-related features like the average time interval between packets, and flag-related features such as the number of packets with the URG flag. For behavioral features, we extract the packet length sequence $L_p = \{l_1, \ldots, l_{N_s}\}$ and direction sequence $D_p = \{d_1, \ldots, d_{N_s}\}$ from the session. $l_i \in \mathbb{N}$ and $d_i \in \{1, -1\}$ are the length and direction of i^{th} packet, and N_s is the number of packets. Additionally, to mitigate the impact of positive and negative signs in the direction sequence on subsequent model training, we encode them as a one-hot matrix with two rows and N_s columns, placing the packet length values of the forward and reverse data flows in separate rows. Ideally, a session should encompass sufficient packets to capture the underlying interaction logic, thereby providing a representative and stable basis for feature characterization.

For a session, the extracted feature representation is denoted as $x = \{x_s, x_e\}$, where x_s represents statistical features and x_e represents behavioral features. Statistical features effectively analyze the overall trends and patterns of the traffic, while behavioral features are stronger in modeling fine-grained traffic patterns. The combination of both provides a more comprehensive perspective for network traffic analysis.

C. Model Training

For the purpose of accurately detecting whether the traffic is drifting and classifying non-drifting traffic, we combine autoencoders with contrastive learning. Autoencoders excel in unsupervised feature learning by capturing the main structure and features of the data through learned compressed representations. Contrastive learning, on the other hand, enhances the model's ability to learn more discriminative features by maximizing the similarity between samples of the same class and minimizing the similarity between samples of different classes [22]. We leverage both the reconstruction capability of autoencoders and the distance metrics derived from contrastive learning to achieve these goals.

1) ACLearner: ACLearner consists of the autoencoder part and the contrastive learning part. Session features x are first processed through the encoder to obtain the latent features z, which are then passed through the decoder to produce the reconstructed session features $\hat{x}$. We compute the Euclidean distance between x and $\hat{x}$ as the reconstruction loss L_r, and f_d denotes Euclidean distance.

$$L_r = f_d(x, \hat{x}) \tag{1}$$

To enhance the distinctiveness and representativeness of the features, we use the latent features z to calculate the contrastive loss L_c. The goal is to minimize the distance between samples of the same class while maximizing the distance between samples of different classes. z^+ represents the latent features of samples belonging to the same class as z, and z^- represents the latent features of samples from different classes. The margin m is used to ensure that if the distance between samples from different classes exceeds m, it does not further influence the training process.

$$L_c = f_d(z, z_i^+) + max(m - f_d(z, z_i^-), 0) \tag{2}$$

The overall loss L is obtained by the weighted sum of reconstruction loss and contrastive loss, and λ_r and λ_c are hyperparameters controlling the weights.

$$L = \lambda_r L_r + \lambda_c L_c \tag{3}$$

For statistical features and behavioral features, we use them to train two separate ACLearners (SACLearner and EACLearner), and then integrate them during decision-making using a bagging approach. This strategy enables each type of feature to concentrate on its specific task, mitigates the impact of sample distribution bias on feature weights, enhances the model's robustness, and reduces the risk of single-point failures. For the SACLearner, which handles statistical features, its autoencoder is composed of several fully connected layers. For the EACLearner, which processes behavioral features, the autoencoder is based on Transformer architecture [23]. The use of the attention mechanism enables it to effectively capture the temporal characteristics of the sessions.

2) Traffic Pre-Cluster: Training ACLearner requires class labels, but using existing labels directly poses several issues. Firstly, fine-grained labels are often missing, and using broad labels such as "normal" or "malicious" can be too coarse. Within each class, there are many more specific subcategories with inherent differences, and grouping samples with significant differences into the same category may negatively affect model convergence. In addition, fine-grained labels may not always be accurate, as determining these labels heavily depends on expert knowledge and establishing a unified classification standard and granularity is challenging. Therefore, we first pre-cluster normal and malicious traffic separately to obtain fine-grained labels for training the ACLearners.

To ensure that ACLearner exhibits generalizability and to mitigate the impact of clustering randomness on performance, we employ more coarse-grained features compared to statistical and behavioral features. Specifically, we divide the packet length range into 10 buckets and count the number of packets within each length range in a session to form a feature vector. The K-means algorithm is then used to cluster normal and malicious traffic into k distinct clusters, respectively. If the number of samples in a cluster is less than ϕ_{cdr} times the average cluster size, this cluster of samples is discarded. ϕ_{cdr} is a threshold coefficient.

D. Classification and Drift Detection

After completing model training, we employ the trained model in conjunction with inter-sample distances and reconstruction loss to determine whether traffic drift has occurred and to classify samples that have not experienced drift into their respective categories.

1) Criteria Establishment: We first establish detection criteria by considering both the distance relationships between samples and the distribution of reconstruction losses to ensure a stable performance. We establish customized criteria for each class rather than using a common baseline derived from all training data, due to the varying distribution and sample sizes across different classes. Specifically, we first calculate the latent features z, including z_s for statistical features and z_e for behavioral features, along with reconstruction losses $L_{r,s}$ and $L_{r,e}$ for each sample in the training set. Then, for each class, we calculate the mean of reconstruction losses $\mu_{L_{r,s}}$ and $\mu_{L_{r,e}}$ alongside their variances $\sigma_{L_{r,s}}$ and $\sigma_{L_{r,e}}$. The class centroids c_s and c_e are computed by averaging the latent features of all samples in each class.

2) Classification and Detection: For a given sample under evaluation, the model first derives intermediate results using statistical and behavioral features separately and then combines these results to produce a final result. Taking the statistical feature x_s as an example, the intermediate result is derived through a two-step process. We first compute its corresponding latent feature z_s and calculate the distances to each class centroid to identify the nearest class k_s. In the second step, we calculate the reconstruction loss $L_{r,s}$ and determine if z_s is a drift relative to the nearest class. If $L_{r,s}$ satisfies $\mu_{L^{k_s}_{r,s}} - n_\sigma \sigma_{L^{k_s}_{r,s}} < L_{r,s} < \mu_{L^{k_s}_{r,s}} + n_\sigma \sigma_{L^{k_s}_{r,s}}$, then with respect to the statistical features, the sample is not a drift and belongs to class k_s. Typically, n_σ is set to 3. $\mu_{L^{k_s}_{r,s}}$ and $\sigma_{L^{k_s}_{r,s}}$ represent the mean and variance of the reconstruction loss for samples belonging to class k_s. Similarly, we employ this approach to detect drift and the class k_e for behavioral features. If both statistical and behavioral indicators show no drift and $k_s = k_e$, the sample remains in the class $k_s(k_e)$ without drift. Otherwise, drift is considered to have occurred.

Compared to directly assessing drift based on distance or overall reconstruction loss distribution, using the reconstruction loss distribution of the nearest class provides a more accurate reflection of whether the sample truly belongs to that class. Distance metrics primarily focus on the similarity between sample features and class centroids but may not fully capture sample anomalies, and distance calculations can be influenced by sample distribution, distance scale, and the complexity of the feature space. In contrast, reconstruction loss directly measures the degree of matching between the sample and the well-trained reconstruction model. The overall reconstruction loss distribution may be affected by the mixed influence of samples from different classes, potentially leading to misjudgment. Our approach can more accurately reflect whether the sample conforms to the characteristics of the class, improve detection sensitivity by capturing specific class patterns, and effectively mitigate interference from sample anomalies or differences between classes.

Furthermore, since different features may exhibit varying sensitivities to different types of traffic, combining the judgments from both feature types using a bagging approach allows us to better accommodate diverse feature information. By cross-validating the results from both feature dimensions, we can effectively enhance the robustness of the detection system and mitigate the impact of failures from any single dimension.

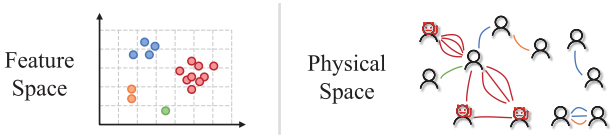

Fig. 2. Schematic diagram of drifting traffic distribution in physical and feature spaces. (red: malicious, others: normal).

E. Drift Identification

For drifting traffic, we aim to leverage their clustering characteristics to automatically determine whether it is malicious, thereby eliminating the reliance on human intervention throughout the process. This process can be triggered by either a specific running time or the volume of samples processed and accumulated. In previous work [19], this step has invariably required human involvement.

Our key observation is that for traffic identified as drift, malicious traffic tends to exhibit concentrated characteristics in both feature space and physical space, whereas benign traffic is more dispersed, as illustrated schematically in Figure 2. Drifting malicious traffic is primarily generated by network attacks. To achieve a successful attack, they often need to make multiple attempts, such as scanning ports, resulting in similar traffic patterns. They are close to each other in feature space (red dots in Figure 2), forming relatively compact clusters. Additionally, to minimize the likelihood of being detected, attackers typically target only a few devices, leading to a concentrated pattern in physical space (red edges), where frequent and similar interactions occur between a limited set of IP addresses. In contrast, in a well-functioning network, drifting benign traffic usually arises from new behaviors triggered by changes in business processes or from the introduction of new users and devices. Since normal users' operations and behaviors are generally heterogeneous, this leads to dispersed patterns in feature space (green and orange dots). Admittedly, some drifting traffic generated by normal users may be concentrated in feature space (blue dots). However, the participants are distinct and dispersed, resulting in a sparse distribution in physical space (blue edges).

Based on these observations, we first cluster the features of the drifting traffic to group together traffic that is similar in the feature space. The feature used here is a concatenation of latent statistical features z_s and behavioral features z_e. We employ DBSCAN for this task as it avoids the need to predetermine the number of clusters and effectively captures high-density patterns. After clustering, for traffic labeled as noise, we consider it to be benign since it is sparse in the feature space.

For other clusters, we evaluate their sparsity in the physical space by constructing a host interaction graph for each cluster. This interaction graph is a multigraph where IP addresses are the nodes, and the interactions between IP pairs are the edges. We then calculate the number of vertices n_v, the number of edges n_e, and the number of connected components n_c in the graph. If the ratio of n_e to n_v exceeds a predefined threshold θ_{ev}, or the ratio of n_e to n_c exceeds θ_{ec}, the traffic

within that cluster is deemed malicious. By combining these two conditions, Argus can detect malicious behavior with highly frequent connections between nodes. For instance, in DDoS attacks or scanning attacks, the attacker attempts high-frequency connections to many targets within a short period, whereas normal network communication does not exhibit such frequent and similar connections within such a brief timeframe [24], [25]. Moreover, Argus can detect malicious behavior that forms one or more tightly connected subgroups, which is typical in lateral movement attacks, where attackers use compromised hosts to move laterally to other hosts, or in botnet activity, where such distributions are observed [26]. If the ratio of n_e to n_v is less than θ_{ev} and the ratio of n_e to n_c is less than θ_{ec}, the traffic in that cluster is considered to be normal.

F. Model Adaptation

To maintain model effectiveness in dynamic environments, we design independent update strategies for drifting and non-drifting data, tailored to their distinct characteristics. Depending on specific needs, the model can be updated using either or both data types. These updates are triggered after one or multiple identification cycles.

1) Update With Non-Drifting Data: Updating with non-drifting data can increase the diversity of known categories, thereby enhancing the model's robustness to subtle changes.

Since updating the model not only requires adjusting model parameters but also involves updating cluster centroids, distance distributions, and reconstruction loss distributions, we adopt a data replay-based update strategy to prevent catastrophic forgetting of previously learned knowledge. Specifically, we randomly select n_{ond} samples from the old training data and mix them with the non-drifting update data X_{nd} to form a new dataset X'_{nd} for model updating. n_{ond} is α_{nd} times the number of samples of X_{nd}, and the value of α_{nd} is usually less than 1.

The overall loss L_{und} during the update process is the weighted sum of the reconstruction loss L_r, the contrastive loss L_c, and the change loss L_{chn}, where $\lambda_{r,und}$, $\lambda_{c,und}$, and λ_{chn} are their respective weights. The calculations for L_r and L_c are the same as those described in Section III-C. The change loss L_{chn} constrains the change in latent features, with z and z' denoting the latent features produced by the old model and the updated model, and is calculated using the Euclidean distance. By minimizing L_{chn}, we ensure a smoother update process, which prevents excessive changes due to overfitting the new data, thereby enhancing the model's robustness, maintaining cluster structure stability, and preserving the consistency of classification and drift detection criteria.

$$L_{und} = \lambda_{r,und} L_r + \lambda_{c,und} L_c + \lambda_{chn} L_{chn} \tag{4}$$

$$L_{chn} = \sum f_d(z, z') \tag{5}$$

Similar to the initial training process, the statistical features and behavioral features are independently used to update the SACLearner and EACLearner, respectively. Subsequently, we update the detection criteria based on the updated models, including c_s, c_e, as well as the corresponding $\mu_{L_{r,s}}$, $\sigma_{L_{r,s}}$, $\mu_{L_{r,e}}$, and $\sigma_{L_{r,e}}$ for each class.

2) Update With Drifting Data: Updating with drifting data can expand the model's coverage of traffic categories, broadening the applicability of the detection method.

When updating the model using drifting data X_{dr}, it is also necessary to include old data. We randomly select n_{odr} old samples to form X_o, and then combine it with X_{dr} to create a new dataset X'_{dr} for model updating. The value of n_{odr} is α_{dr} times the number of samples in X_{dr}. Note that the classes of samples in X_o do not overlap with those in X_{dr}. During the drift identification process (Sec III-E), each cluster obtained by DBSCAN is treated as an independent class. However, the number of clusters is often large, and each cluster contains a relatively small number of samples, leading to issues of data imbalance and insufficient sample representativeness, which are detrimental to model updating. To address this issue, we performed a merging operation on these clusters, repeatedly merging the two clusters with the closest centroids until only n_{cl} clusters remained.

The loss function during the update process, L_{udr}, similarly consists of the reconstruction loss L_r, the contrastive loss L_c, and the change loss L_{chd}, and $\lambda_{r,udr}$, $\lambda_{c,udr}$, λ_{chd} are corresponding weights. However, L_r and L_c are computed using all of the data X'_{dr}, whereas L_{chd} is calculated only from the data in X_o, as the drifting data does not conform to the distribution of the old model.

Additionally, L_{chd} measures the change in distance between the latent features of samples in the updated model and the old cluster centroids c of their classes, instead of directly measuring latent feature changes. This is a more flexible constraint, which is crucial when updating with drifting data, as the process involves the introduction of new classes. Adequate space must be reserved in the feature space to effectively distinguish and represent new classes.

$$L_{udr} = \lambda_{r,udr} L_r + \lambda_{c,udr} L_c + \lambda_{chd} L_{chd} \tag{6}$$

$$L_{chd} = \sum_{z \in X_o} |f_d(z, c) - f_d(z', c)| \tag{7}$$

The updates are performed separately on SACLearner and EACLearner similarly. The method for updating the detection criteria is the same as when updating with non-drifting data.

IV. Evaluation

A. Experimental Setup

1) Datasets: Argus aims to accurately classify known traffic, detect and identify drifting traffic, and adaptively update in environments with evolving traffic patterns. To evaluate the performance of our method, we reorganize both publicly available datasets (CICIDS2018 [27], CICIDS2017 [28], MCFP [29] and ISCX-NonVPN [30]) and our own collected dataset (NormTI) to create two datasets: CICIDD and MalReal. Each dataset is divided into multiple subsets for experiments to assess how Argus performs in the presence of varying degrees and types of drifts.

This reorganization establishes a controlled evaluation framework, where the comparative performance across subsets

serves as a reliable proxy for quantifying drift detection efficacy. Specifically, we construct these subsets to span a spectrum of drift scenarios: from stable baselines with unavoidable minor fluctuations to extreme drifts triggered by novel attacks or environmental changes. By analyzing the performance variance across these controlled scenarios, we can rigorously evaluate the sensitivity and robustness of drift detection modules.

A detailed description of the subsets and their corresponding drift characteristics is provided below.

CICIDD. This dataset contains common network intrusion traffic, derived from the publicly available CICIDS2018 and CICIDS2017 datasets. We reorganize this data into three subsets (C-D1, C-D2, C-D3). The normal traffic in each subset is sourced from CICIDS2018 and partitioned by date, thereby incorporating natural drift resulting from temporal shifts. The malicious traffic in C-D1 and C-D2 is sourced from the CICIDS2018 dataset.

- **C-D1** includes the following types of malicious traffic: Bot, DDoS LOIC HTTP, DoS GoldenEye, DoS slowhttp, and FTP Bruteforce.
- **C-D2** includes a broader range of attacks compared to C-D1. It features variants of known attacks (DoS Hulk and SSH Bruteforce) implemented with different techniques, as well as new attack categories (XSS Bruteforce, Web Bruteforce, and SQL Injection) that are entirely different from C-D1, thereby introducing a distinct degree of drift.
- **C-D3** comprises malicious traffic from the CICIDS2017 dataset. While it covers the same attack categories as the baseline (e.g., Botnet, DoS, and Web attacks), the underlying traffic patterns exhibit significant deviation due to the temporal gap and distinct network environments. This simulates a realistic scenario of long-term pattern evolution, representing a more significant degree of drift compared to short-term variations.

MalReal. This dataset primarily includes traffic generated by malware, consisting of data from the publicly available MCFP and ISCX-NonVPN datasets, as well as the self-collected NormTI dataset. We reorganize this data into four subsets (M-D1, M-D2, M-D3, M-D4).

- The normal traffic in **M-D1**, **M-D2**, and **M-D3** comes from the NormTI dataset. It includes normal user traffic captured over a period of three months in 2024 from a subnet within a research institution, involving nearly 1,000 devices. Similar to CICIDD, this traffic is partitioned chronologically, thereby introducing natural drift resulting from temporal shifts.
- The normal traffic in **M-D4** comes from the ISCX-NonVPN dataset, which includes network traffic generated by real-world activities such as chat, file transfer, and video streaming. This traffic differs significantly from the normal traffic in M-D1 in terms of network environment and user behavior patterns, representing severe drift in normal traffic.

  The malicious traffic in this dataset comes from the publicly available MCFP dataset, which includes traffic generated by various malware executions. Specifically:
  - **M-D1** contains malicious traffic generated by botware (Trickbot, Emotet) and ransomware (Dridex).
  - **M-D2** extends the M-D1 baseline by incorporating traffic generated by an additional botware variant (Zeus). This represents a drift scenario driven by new malware variants. It is important to note that even within the same malware category, variations in tool versions, execution procedures, or targets lead to distinct traffic patterns, contributing to the overall drift.
  - **M-D3** represents a high-drift scenario that builds upon M-D2. In addition to the existing traffic, it introduces new malware categories and variants, including miner (MinerTrojan), Spyware (Trickster, CCleaner), and distinct Ransomware strains (WannaCry).
  - The malicious traffic in **M-D4** is consistent with that in M-D2 and is primarily used to test the drift in normal traffic.

To facilitate rigorous evaluation, each subset within the CICIDD and MalReal datasets is pre-partitioned into training and testing sets. Model training and adaptive updates are strictly confined to the training sets, whereas all performance evaluations are conducted exclusively on the testing sets. In our experimental setup, the training sets of C-D1 and M-D1 serve as the baselines for initial model training.

Accordingly, Table I summarizes the drift sources and drift levels of normal and malicious traffic in the remaining subsets relative to these two baselines. A higher drift level indicates a greater deviation from the baseline distribution, implying a higher difficulty for the detection model.

2) Metric: We use accuracy (Acc), recall (Rec), precision (Pre), and F1-score (F1) as evaluation metrics to assess whether malicious traffic is accurately detected. Moreover, the drift ratio (DR) and the non-drift ratio (NDR) are employed to assess the percentage of samples identified as drift or non-drift relative to the total number of samples.[2]

3) Parameter Setting: For the CICIDD dataset, model training uses $k = 3$, $\phi_{cdr} = 0.05$, and $m = 10$. Loss weights λ_c and λ_r are 10 and 0.01. Drift detection is set at $n\sigma = 3$. Drift identification utilizes DBSCAN ($Eps = 2.0$, $MinPts = 10$) with thresholds $\theta_{ec} = 5$ and $\theta_{ev} = 10$. Adaptation parameters α_{nd} and α_{dr} are 0.1. Update weights for contrastive, reconstruction, and constraint losses are 1, 0.5, and 0.001, respectively. For the MalReal dataset, training parameters are $k = 5$, $\phi_{cdr} = 0.05$, $m = 10$, with $\lambda_c = 1$ and $\lambda_r = 0.01$. Drift detection uses $n\sigma = 3$. Identification employs $Eps = 1.0$, $MinPts = 10$, $\theta_{ec} = 5$, and $\theta_{ev} = 10$. Adaptation uses $\alpha_{nd} = \alpha_{dr} = 0.1$. Update process weights match those of CICIDD (1, 0.5, and 0.001). Detailed sensitivity analyses for these parameters are provided in Section IV-F.

4) Baseline: We consider three categories of baseline methods, including those aimed at improving detection accuracy for standard network malicious traffic (the supervised method FS-Net [4] and the semi-supervised method CPS-Guard [31]), those robust methods focused on enhancing generalization

[2]Due to the inconsistency in label granularity and the fact that differing labels do not necessarily indicate drift, especially for normal samples, we do not use metrics such as recall and precision specifically for detecting drift, as is common in other drift detection studies.

TABLE I
DRIFT SOURCES AND DRIFT LEVELS OF DATASETS RELATIVE TO C-D1 AND M-D1

	Normal		Attack	
	Drift Source	Drift Level	Drift Source	Drift Level
C-D1	Baseline	◴	Baseline	◴
C-D2	Temporal Shift	◴	New Attack Categories, Variants	◐
C-D3	Temporal Shift	◴	Long-term Pattern Evolution	◕
M-D1	Baseline	◴	Baseline	◐
M-D2	Temporal Shift	◴	New Malware Variants	◐
M-D3	Temporal Shift	◴	New Malware Categories, Variants	◕
M-D4	Environment, User Behavior	●	New Malware Variants	◐

- More black in the circles indicates higher drift.

TABLE II
PERFORMANCE COMPARISON OF ARGUS AND BASELINES ON THE CICIDD DATASET (%)

	Method	Acc	Rec	Pre	F1
C-D1	CPS-Guard	80.69	40.91	93.57	56.93
	FS-Net	98.36	**99.96**	95.04	97.43
	ACID	**99.72**	99.14	99.72	**99.55**
	CADE	69.42	99.91	50.49	67.08
	AOC-IDS	90.77	70.43	**99.95**	82.63
	Argus	98.38	99.86	95.17	97.46
C-D2	CPS-Guard	83.45	32.57	89.38	47.34
	FS-Net	91.05	67.86	91.33	77.86
	ACID	92.53	67.80	**99.97**	80.80
	CADE	67.82	75.17	39.77	52.02
	AOC-IDS	83.88	33.91	90.88	49.41
	Argus	**98.28**	**99.33**	93.65	**96.41**
C-D3	CPS-Guard	96.67	71.05	79.57	75.07
	FS-Net	93.36	21.66	58.10	31.55
	ACID	93.94	14.24	**99.65**	24.92
	CADE	54.16	0.84	0.15	0.25
	AOC-IDS	93.85	14.60	89.87	25.12
	Argus	**98.32**	**89.12**	87.35	**88.22**

TABLE III
PERFORMANCE COMPARISON OF ARGUS AND BASELINES ON THE MALREAL DATASET (%)

	Method	Acc	Rec	Pre	F1
M-D1	CPS-Guard	69.26	9.18	77.83	16.43
	FS-Net	97.86	99.72	94.12	96.84
	ACID	**99.92**	99.78	**99.99**	**99.88**
	CADE	96.65	**100.00**	90.77	95.16
	AOC-IDS	90.29	99.98	77.21	87.13
	Argus	97.75	98.77	94.63	96.66
M-D2	CPS-Guard	59.05	26.32	93.09	41.04
	FS-Net	95.53	94.71	96.95	95.82
	ACID	86.35	74.80	**99.99**	85.58
	CADE	78.45	62.94	61.56	62.24
	AOC-IDS	92.97	**99.99**	88.52	93.91
	Argus	**96.24**	94.62	98.37	**96.46**
M-D3	CPS-Guard	59.52	11.91	78.13	20.67
	FS-Net	93.87	89.79	96.12	92.85
	ACID	81.92	59.21	**99.94**	74.36
	CADE	83.23	**93.39**	59.85	72.95
	AOC-IDS	73.77	88.12	65.05	74.85
	Argus	**94.13**	90.65	95.87	**93.19**
M-D4	CPS-Guard	11.43	8.11	88.70	14.86
	FS-Net	95.39	**96.99**	98.15	97.57
	ACID	75.57	75.09	99.05	85.42
	CADE	83.30	82.66	**99.79**	90.42
	AOC-IDS	33.27	96.93	30.97	46.94
	Argus	**95.98**	96.59	99.18	**97.86**

in the presence of varying traffic (ACID [11]), and methods capable of detecting concept drift (CADE [16] and AOC-IDS [32], with AOC-IDS supporting adaptive updates).

B. Detection Performance

1) Comparison With Baselines: Tables II and Table III present the performance of Argus and other methods[3] across various subsets of the CICIDD and MalReal datasets. These results are obtained by training on the C-D1 and M-D1 training sets and testing on the corresponding test sets of each subset. It can be observed that our method, Argus, outperforms the other methods, with F1 scores consistently above 93% across all subsets of MalReal. Moreover, Argus demonstrates significantly more stable performance under varying degrees and types of drift compared to the baselines, with F1 score fluctuations of only 9.24% across the CICIDD subsets and 4.67% across the MalReal subsets.

Supervised network malicious traffic detection methods (such as FS-Net and ACID) perform well when the training traffic and testing traffic follow the same distribution. However, their performance may degrade significantly in the presence of drift. For example, FS-Net achieves an F1 score of 97.43% on C-D1, but only 31.55% on C-D3, indicating its inability to handle drift. Although ACID attempts to enhance robustness by extending features and improving the method, its effectiveness is limited, only being suitable for slight drifts and unable to handle new types of traffic. For instance, its F1 score drops by 25.52% on M-D3 with more significant drifts.

CPS-Guard performs poorly on MalReal, particularly with a low recall. CPS-Guard's initial training is conducted solely on normal traffic, and it adaptively determines decision thresholds based on the distribution of test data. This method performs

[3]Since CADE is primarily a drift detector, its output was post-processed for the malicious traffic detection performance evaluation: non-drifting samples were classified as the category of their nearest cluster, while drifting samples were labeled according to the dataset organization rules (normal traffic for M-D4, and malicious traffic for all other datasets). All other methods, including Argus, are evaluated directly using their primary malicious traffic detection output.

TABLE IV
DETAILED PERFORMANCE OF ARGUS AND CADE ON THE CICIDD DATASET AND THE MALREAL DATASET (%)

Dataset	Argus			CADE	
	DR All / Normal / Malicious	Non-Drifting Acc / F1	Drifting Acc / F1	DR All / Normal / Malicious	Non-Drifting Acc / F1
C-D1	2.89 / 2.99 / 2.67	99.97 / 99.95	44.97 / 50.42	2.80 / 2.73 / 2.97	70.47 / 67.81
C-D2	9.59 / 2.59 / 32.74	99.95 / 99.85	82.57 / 89.95	19.46 / 23.19 / 7.11	82.16 / 68.73
C-D3	8.67 / 2.13 / 94.67	99.58 / 0.00	84.94 / 90.63	10.48 / 11.27 / 0.00	60.49 / 0.33
M-D1	12.26 / 8.64 / 19.65	99.85 / 99.74	82.78 / 85.22	3.35 / 4.99 / 0.00	96.65 / 95.16
M-D2	14.53 / 8.60 / 19.56	97.16 / 97.14	90.78 / 93.74	14.74 / 15.45 / 12.94	87.73 / 72.96
M-D3	22.47 / 9.62 / 38.67	97.64 / 96.53	82.03 / 87.92	17.77 / 19.34 / 12.85	97.43 / 94.87
M-D4	24.75 / 65.85 / 22.73	97.99 / 98.97	89.84 / 93.98	13.71 / 57.27 / 11.57	94.43 / 96.52

well when the difference between attack traffic and normal traffic is significant, as evidenced by its F1 score of 75.07% on C-D3, second only to Argus. However, its performance deteriorates when normal and malicious traffic are similar, as seen in its poor performance on M-D1, where other supervised methods can learn the differences from labeled data.

AOC-IDS uses Gaussian distributions to model each class in the training data, classifying and detecting anomalies in the test data. It faces similar challenges, struggling to handle drift in normal traffic, with an F1 score of only 46.94% on M-D4.

CADE's overall performance on CICIDD is lower than on MalReal, with generally lower precision on the CICIDD datasets, suggesting a higher rate of misclassification of normal traffic. One possible reason is that all normal traffic is treated as a single class, failing to effectively model intra-class variation. Besides, the issue of data imbalance significantly impacts the contrastive learning method used by CADE. Another factor is our post-processing strategy for drifting samples. As CADE lacks an automated mechanism to determine the category of drift, uniformly labeling these samples as normal or malicious based on dataset organization introduces an increased risk of false positives. This also highlights the necessity of automated drift identification, as detection is the ultimate goal, not just drift observation. Detailed results regarding CADE's drift detection and non-drifting sample classification will be further analyzed in Section IV-B.2.

Although AOC-IDS, CADE, and our Argus are all based on contrastive learning and autoencoder architectures, Argus demonstrates better performance than the others. This is attributed to our integration of statistical and behavioral features, the design of AClearner, and the collaboration across multiple stages, all contributing to the overall enhancement of performance.

2) Detailed Performance for Non-Drifting and Drifting Traffic: The outstanding performance of Argus is attributed to the combined contributions of its traffic classification and drift identification components. Table IV provides a detailed comparison of the performances of Argus and CADE,[4] including drift ratios (DR) of different splits of datasets (entire testing set, only normal traffic or only malicious traffic) as well as the accuracy and F1 score for malicious traffic detection on data identified as drift and non-drift.

[4]CADE does not determine the category of samples identified as drift, so we only compare the performance on non-drifting samples. AOC-IDS does not provide explicit results on whether a sample is a drift sample, so we do not include comparisons with it in this section.

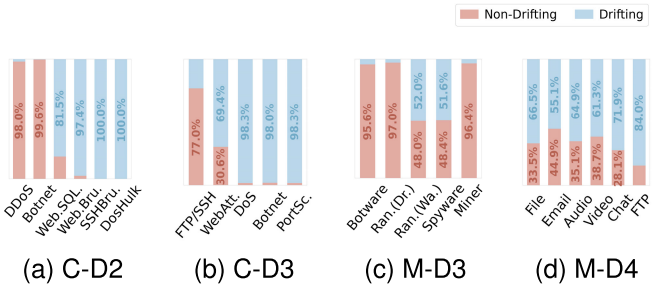

Fig. 3. Ratio of non-drifting and drifting samples for each category.

Argus is capable of appropriately identifying drift in both normal and malicious traffic, and this is evidenced by the consistency between the drift ratios across different subsets identified by Argus (Table IV) and the drift levels established during subset partitioning (Table I). For instance, in the CICIDD dataset, the drift level of malicious traffic increases from C-D1 to C-D3, and Argus's drift ratio for malicious traffic in these subsets rises from 2.67% to 94.67%.

Furthermore, Figure 3 visually presents the ratios of non-drifting and drifting samples identified by Argus for various categories, illustrating the module's effectiveness in distinguishing between different types and degrees of concept drift. In C-D2, known categories like DDoS and Botnet remain stable, with a non-drifting proportion greater than 98%, while newly introduced categories such as WebBruteforce and DoS Hulk are largely marked as drifting. Correspondingly, in M-D3, Argus differentiates known malware (Botware) from high-drift new variants (e.g., Spyware with 51.6% drifting), validating its detection of novel variants. Furthermore, Argus is sensitive to temporal and environmental drift. The C-D3 subset shows high drift in known Botnet traffic, with 98% of samples identified as drift. Similarly, when Argus evaluates M-D4, all normal traffic categories are detected to exhibit significant drift, exceeding 50% in every case, with FTP traffic showing the highest detected drift ratio at 84%, underscoring the severity of benign environmental change.

Argus also demonstrates satisfactory performance in detecting malicious traffic within both drifting and non-drifting traffic. For non-drifting traffic, Argus classifies the traffic based on the class closest to the cluster center in the latent feature

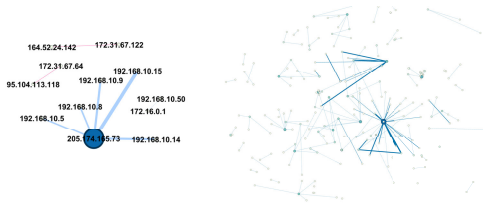

Fig. 4. Distribution of drifting traffic clustered in feature space within physical space (left identified as malicious by argus, right as normal).

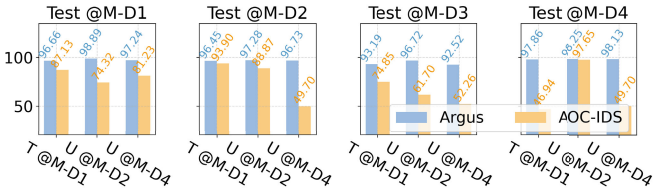

Fig. 5. Comparison of F1 for argus and AOC-IDS on the malreal dataset before and after adaptations. (T: train, U: update).

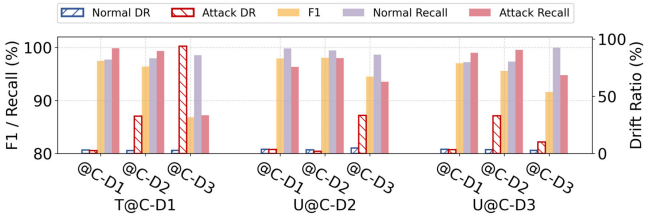

Fig. 6. Impact of model adaptation on drift ratio and F1 for argus on the CICIDD dataset.

space. Across these datasets, Argus achieves a detection accuracy of over 97% for non-drifting traffic, outperforming that of CADE. In most cases, non-drifting traffic accounts for a large proportion of the total traffic, so the detection performance on non-drifting traffic is critical. For drifting traffic, Argus leverages its clustering features in both the feature space and physical space to determine whether the traffic is malicious. In almost all cases, Argus achieves an F1 score of over 85% for classifying drifting traffic, enhancing Argus's applicability. Moreover, this process is labor-free, boosting its efficiency.

In comparison, CADE's performance is not as favorable. For example, in C-D3, the subset with the highest degree of drift in CICIDD, CADE incorrectly concludes that no drift exists within the malicious traffic. Furthermore, it fails to accurately detect these "non-drifting" malicious traffic, as reflected by its low F1 score of just 0.33% for non-drifting traffic.

Figure 4 shows the distribution of two groups of traffic clustered in feature space within physical space, further validating the effectiveness of the drift identification module of Argus. Malicious traffic come from a Botnet attack, and they cluster in both feature and physical spaces due to targeted attacks. Normal traffic, though clustered in feature space, is dispersed in physical space, likely due to network environment changes.

C. Adaptation Performance

The ultimate goal of addressing concept drift is to prevent model performance degradation, thereby ensuring the model's effectiveness and accuracy in ever-changing environments. We utilize the C-D2, C-D3, M-D2, and M-D4 datasets to update the models initially trained on the C-D1 and M-D1 datasets, intending to validate the effectiveness of the model update module in Argus. Note that the updates are executed using the training subsets corresponding to each respective dataset, with performance validated on the corresponding test sets, ensuring no overlap between training and test data. Both non-drifting and drifting data updates are performed.

Figure 5 compares the F1 of Argus and AOC-IDS across different MalReal sub-datasets, both before and after the updates. It can be observed that after updating with M-D2 and M-D4, Argus's performance on M-D1 remains stable, and its performance on M-D2 and M-D4 improves. This indicates that Argus possesses the capability to continue learning and adapting to new knowledge without forgetting previously acquired knowledge, effectively addressing the challenges posed by concept drift. Furthermore, after updating with M-D2, Argus also shows improved performance on M-D1 and M-D3. This may be because the update process using non-drifting data increases data diversity, thereby enhancing its generalization capability. In contrast, after updating with M-D2, the performance of AOC-IDS on the M-D2 testing set declines to 88.87%, which is lower than its initial performance when trained on M-D1. Its performance on M-D1 and M-D3 also decreases by at least 5.9% and 13.15%, respectively. This is primarily due to AOC-IDS's reliance on the distribution of latent features during detection, coupled with the lack of effective constraints during the update process. When updated with drifting data, AOC-IDS struggles to balance new and old knowledge, resulting in considerable deviation from the original model.

Figure 6 provides a detailed analysis of the joint trend of drift ratio and performance (overall F1 and recall for both normal and attack traffic) for Argus on the CICIDD dataset before (T@C-D1) and after updates (U@C-D2 and U@C-D3). It can be seen that after updating with the C-D2 or C-D3 datasets, the corresponding attack drift ratio decreases while the F1 increases. Notably, the recall for attack traffic exhibits a substantial improvement, demonstrating that the update mechanism in Argus effectively enhances the model's adaptability to new classes and enables it to recognize emerging categories. Furthermore, after updating with C-D2 or C-D3, the performance on C-D1 does not experience any significant degradation. The recall for normal traffic on C-D1 even shows an improvement after the C-D2 update. In addition, an interesting phenomenon is observed: after updating with C-D2, the drift ratio for C-D3 also decreases significantly. This may be attributed to that the new attack traffic categories introduced in C-D2 overlap with those in C-D3. Although they originate from different datasets, C-D2 effectively carves out a new space in the feature space for attack traffic, which could cover some of the traffic in C-D3, thereby positively impacting the detection of malicious traffic in C-D3.

TABLE V
RUNTIME AND MEMORY OVERHEAD OF ARGUS (30,000 SESSIONS)

	I.TFE	II.MT	III.CDD	IV.DI	V.MA
Time (s)	17.72	4.68	6.26	21.73	6.91
Memory (MB)	2772.53	69.17	375.98	201.91	41.57

- MT denotes Model Training. Other modules are abbreviated accordingly.
- MT and MA both execute for one epoch.

D. Efficiency and Overhead Analysis

To verify the practical feasibility of Argus, we evaluate the processing latency and memory consumption across all stages. Table V presents the quantitative results for processing 30,000 sessions. These sessions were randomly sampled from the datasets to reflect real-world traffic distribution, with an average duration of 33s, 244 packets, and 120,955 bytes per session. The results indicate that the Classification and Drift Detection module, which handle real-time traffic, operate at high speeds to meet the requirements of high-speed network environments. Other steps are executed asynchronously or offline, ensuring they do not block the primary detection path.

Regarding memory usage, the overhead is primarily concentrated in the initial feature extraction stage where raw PCAP files are processed. Once in the subsequent stages, the system operates only on extracted features or low-dimensional latent representations, which significantly reduces the memory usage. Furthermore, the additional memory cost during model updates stems only storing a small fraction of historical features. Since the replay ratio is kept low, this overhead remains minimal, ensuring the system's robustness in resource-constrained environments.

E. Ablation Study

To validate the contributions of each component within Argus, we conduct comprehensive ablation experiments on two datasets. In these experiments, we either remove or replace various components within each stage of Argus to assess their impact. Table VI presents the results of these ablation studies.

1) Ablation on Traffic Feature Extraction: During the feature extraction phase, Argus extracts statistical features and behavioral features from the traffic data to train SACLearner and EACLearner separately. The final decision-making process involves the joint contribution of these two models. We evaluate the performance of using only SACLearner (w/ SAC) and only EACLearner (w/ EAC) here to further analyze the impact of each feature. The experimental results show that Argus achieves more stable and superior performance by combining both types of features, which not only expands the coverage of traffic types but also reduces the risk of single points of failure. Conversely, utilizing only a single feature leads to relatively unstable performance, occasionally leading to significant declines. For example, on C-D2, using only behavioral features (w/ EAC) yields an F1 of 77.94%, representing an approximately 20% drop compared to C-D1.

2) Ablation on Model Training: In real-world scenarios, precise fine-grained labels are often unavailable, especially for normal traffic. Therefore, during training, we use only coarse labels indicating whether the traffic is normal or malicious. However, considering the impact of intra-class sample variability, Argus incorporates a pre-clustering mechanism to generate fine-grained labels. We apply pre-clustering to both normal and malicious traffic. To validate the necessity and effectiveness of pre-clustering, we conduct experiments where we do not cluster the normal traffic (w/o NPC), treating all normal traffic as a single class, and where we do not cluster the malicious traffic (w/o MPC), using the fine-grained labels available in the dataset for malicious traffic. The experimental results suggest that pre-clustering normal traffic may improve performance, particularly on the CICIDD dataset, where the F1 increases by up to 14.42%. The consistency of intra-class samples is crucial for the model to accurately capture the characteristics of each class, and Argus's pre-clustering effectively enhances class cohesion, thereby positively influencing training. Moreover, compared to using the true fine-grained labels for malicious traffic, Argus exhibits only a maximum decrease of 1.91% in F1, demonstrating that our pre-clustering method is effective and achieves performance close to that of training with true labels. Pre-clustering enhances Argus's generalizability, particularly when fine-grained labels are lacking or inconsistent.

3) Ablation on Classification and Drift Detection: Argus identifies drifting traffic by evaluating whether the reconstruction loss of the traffic aligns with the reconstruction loss of its nearest class. We compare this approach with a distance-based method (w/ Dist), which assesses whether the distance between a sample and its nearest class centroid matches the distribution of distances within that class. It performs well on C-D1, achieving an F1 score 1.03% higher than Argus. However, on C-D2 and C-D3, its F1 scores are more than 10% lower than Argus. The stability of using distance directly to detect drift is relatively poor, as its performance is significantly influenced by the intra-class sample distribution and the model's fit, particularly for classes not encountered during training. We also compare Argus with a method based on overall reconstruction loss (w/ AllRecon), which checks if the traffic's reconstruction loss aligns with the distribution of overall reconstruction losses. Using overall reconstruction loss leads to a performance drop across all datasets, as it overlooks the differences between classes, and the overall reconstruction loss distribution can be skewed by specific class distributions. Argus, by combining reconstruction loss with class specificity, better balances the influence of different features, resulting in superior performance in detecting drifting traffic.

4) Ablation on Drift Identification: Argus considers traffic that exhibits clustering in both feature space and physical space within drifting samples as malicious. In the ablation experiments, we compare two simplified methods: one that disregards the clustering characteristics in both feature and physical spaces, and classifies all drifting traffic as malicious (w/o FPS), based on the assumption that normal traffic in a well-functioning network typically does not exhibit drift [33]; and another that ignores clustering in physical space (w/o PS), treating the noise identified by DBSCAN as normal traffic and the remaining clustered traffic as malicious, based on that drifting malicious traffic tends to be more concentrated in feature space compared to normal traffic, as discussed

TABLE VI
ABLATION STUDY OF KEY COMPONENTS IN ARGUS ON THE CICIDD AND MALREAL DATASETS (%)

Stage	Dataset	CICIDD			MalReal			
		C-D1 Acc / F1	C-D2 Acc / F1	C-D3 Acc / F1	M-D1 Acc / F1	M-D2 Acc / F1	M-D3 Acc / F1	M-D4 Acc / F1
Traffic Feature Extraction	w/ SAC	99.03 / 98.46	98.27 / 96.26	96.11 / 67.48	82.97 / 79.37	87.70 / 89.40	78.34 / 76.82	91.66 / 95.29
	w/ EAC	98.38 / 97.45	91.18 / 77.94	97.99 / 85.78	97.04 / 95.67	91.23 / 89.58	91.35 / 89.74	95.05 / 97.39
Model Training	w/o NPC	97.76 / 96.52	95.06 / 81.99	96.56 / 79.64	97.97 / 96.98	97.66 / 97.84	92.56 / 91.19	96.92 / 98.38
	w/o MPC	98.69 / 97.94	98.56 / 96.97	98.66 / 90.13	98.36 / 97.55	96.64 / 96.84	93.31 / 92.04	95.98 / 97.86
Classification and Drift Detection	w/ Dist	99.05 / 98.49	93.27 / 84.47	96.23 / 70.26	94.99 / 92.81	95.55 / 95.84	89.64 / 87.65	96.46 / 98.13
	w/ AllRecon	98.33 / 97.39	98.25 / 96.31	97.49 / 82.70	95.28 / 93.27	95.84 / 96.13	91.62 / 90.26	95.23 / 97.47
Drift Identification*	w/o FPS	31.97 / 45.14	79.25 / 88.42	77.15 / 87.10	52.70 / 69.03	72.85 / 84.30	76.16 / 86.46	87.52 / 93.34
	w/o PS	28.76 / 44.68	79.16 / 88.24	76.15 / 86.04	79.76 / 83.16	88.17 / 92.15	80.85 / 87.29	89.74 / 93.92
	Argus-D	44.97 / 50.42	82.57 / 89.95	84.98 / 90.63	82.78 / 85.22	90.78 / 93.74	82.03 / 87.92	89.84 / 93.97
Model Adaptation^{+}	w/o Cons	98.53 / 97.68	97.00 / 93.45	98.63 / 90.06	97.83 / 96.75	95.94 / 96.11	93.60 / 92.48	94.83 / 97.23
	Argus-U	98.11 / 97.04	97.86 / 95.57	98.90 / 91.61	97.08 / 97.24	96.53 / 96.73	93.79 / 92.52	96.47 / 98.13
Argus (Default)		98.38 / 97.46	98.28 / 96.41	97.44 / 88.22	97.75 / 96.66	96.23 / 96.46	94.13 / 93.19	95.98 / 97.86

- * indicates that the result represents the performance on drifting traffic, while the absence of * denotes the overall performance.
- $^{+}$ indicates that the result represents the performance after updates, while the absence of $^{+}$ denotes the performance from initial trained model .

in Section III-E. Table VI compares their performance with Argus (Argus-D) in identifying the categories of traffic already determined to be drifting. The combination of both dimensions is crucial for enhancing overall performance, especially in the MalReal dataset. In the CICIDD dataset, the approach that considers only feature space (w/o PS) performs slightly worse than classifying all drifting traffic as malicious (w/o FPS). However, this does not imply that clustering in feature space is an ineffective characteristic. On the contrary, Argus achieves improved performance by first identifying clustering in feature space and then analyzing it in physical space, effectively combining these two dimensions.

5) Ablation on Model Adaptation: In Argus, we adopt an updating approach with constrain to prevent significant changes in the model and clusters during the update process, which could negatively impact performance. We update the models initially trained on C-D1 and M-D1 using C-D3 and M-D4, respectively, and compare the performance of Argus's default constrained update method (Argus-U) with the unconstrained method (w/o Cons). Except for the C-D1 dataset, the constrained approach consistently outperforms the unconstrained one, highlighting the importance of incorporating constraints during updates.

F. Parameters Analysis

We further study the impact of different parameter values on the performance of Argus. Below, we analyze the parameters involved in each module separately.

1) Parameters Analysis on Model Training: In the model training phase, we investigate the effects of the pre-clustering cluster number k, the proportion of discarded class samples ϕ_{cdr}, and the margin m in ACLearner, along with the weights of contrastive loss and reconstruction loss, λ_c and λ_r, on Argus' performance. We focus on analyzing non-drift ratio (NDR) and the accuracy in classifying these non-drifting traffic (ND-Acc), rather than evaluating overall performance, to avoid interference from parameter settings during the drift identification process. Experiments are conducted on the M-D2 and M-D4 datasets, which include drifting malicious traffic and drifting normal traffic, respectively.

In the pre-clustering process, as shown in Figure 7a, model performance is optimal when the value of k is moderate (i.e., $k = 5$). Smaller values of k do not provide sufficient intra-class specificity, while larger values of k tend to partition similar samples into separate clusters. Regarding ϕ_{cdr}, Figure 7b shows that when $\phi_{cdr} = 0.01$, ND-Acc for M-D4 normal traffic is relatively poor, as this setting retains some low-sample categories, which, as independent classes for training, are insufficient and prone to misclassification.

In ACLearner, the margin m represents the minimum required distance between samples of different classes, and the analysis results are shown in Figure 7c. When m is small, the model's ability to differentiate between samples of different classes decreases, as evidenced by a significant drop in accuracy for normal traffic in M-D4. Additionally, a smaller m reduces the inter-class distance, which also shortens the distance among samples within the same class, making the model more likely to classify unknown samples as drifting. Conversely, a larger margin led the model to classify more samples similar to the training distribution as non-drift samples; for example, in M-D2, the proportion of non-drift normal traffic samples increases with larger margins.

Besides, as shown in Figure 7d, the weights of reconstruction loss and contrastive loss generally had little effect on model performance. However, when the weight of reconstruction loss is too high, it significantly impacts performance. Given that reconstruction loss has an inherently large value, assigning it a higher weight diminishes the effect of contrastive learning, resulting in poor separation of samples from different classes in feature space.

2) Parameters Analysis on Drift Detection and Classification: The drift detection phase determines whether the reconstruction loss of a new sample's statistical and behavioral

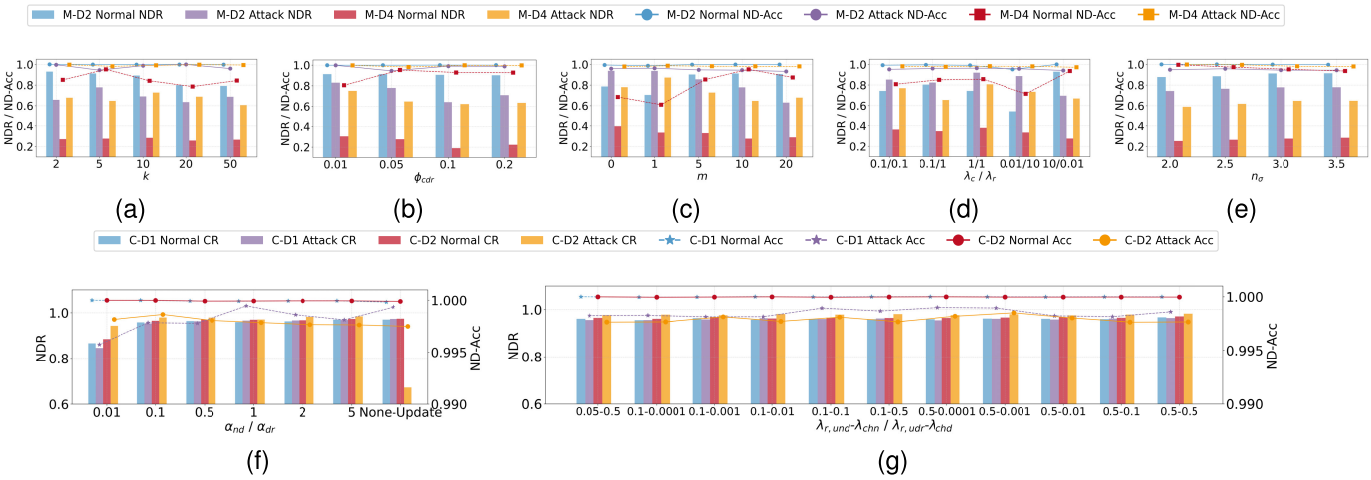

Fig. 7. Parameters analysis of model training and model adaptation modules.

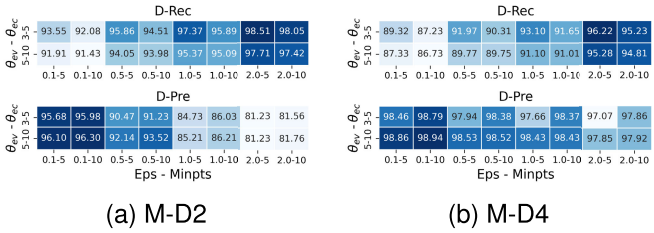

Fig. 8. Parameters analysis of drift identification module.

features falls within the n_σ range of the training data. Figure 7e illustrates NDR and ND-Acc on M-D2 and M-D4. It is observed that as the value of $n\sigma$ increases, the criteria for identifying non-drifting data become more relaxed, leading to an increase in NDR and a slight decrease in ND-Acc. By leveraging contrastive learning within the ACLearner training phase, we achieve compact class representations, which allows the n_σ-based threshold to accurately identify drifting samples through their reconstruction loss.

There is an inherent trade-off between the drift detection rate and non-drift classification accuracy, and the results indicate that the overall impact remains marginal. Furthermore, the subsequent drift identification module automatically analyzes samples flagged as drifting, thereby mitigating the manual burden typically associated with tuning this trade-off.

3) Parameters Analysis on Drift Identification: In the drift identification phase, the key parameters include the two DBSCAN parameters (Eps and MinPts) and two thresholds, θ_{ec} and θ_{ev}, used to determine whether a cluster should be classified as malicious. Due to the interdependence among these parameters, we present the precision (D-Pre) and recall (D-Rec) performance for drifting traffic of Argus under various parameter combinations on M-D2 and M-D4 in Figure 8. We can observe that as Eps increases, D-Rec improves due to larger clusters incorporating more data points. Conversely, as MinPts increases, D-Rec declines, as higher MinPts values result in more data points being classified as noise, reducing the number of positively identified samples. Increasing the thresholds θ_{ec} and θ_{ev} also leads to a slight decrease in recall. D-Pre and D-Rec exhibit a negative correlation, which necessitates a trade-off in parameter selection based on specific requirements in practical applications to balance detection accuracy and the risk of missed detections.

4) Parameters Analysis on Model Adaptation: We use C-D2 to update the model initially trained with C-D1 to study the impact of parameters during the adaptation process. Figure 7f illustrates the variations in non-drift ratio and ND-Acc across the C-D1 and C-D2 datasets. As shown, updating with C-D2 effectively increases the non-drift ratio of the relevant test data, enhancing adaptability to new types of data. Moreover, when the values of α_{nd} and α_{dr} exceed 0.1, the non-drift ratio and ND-Acc on the C-D1 dataset remain almost unchanged from their pre-update levels, indicating that a small amount of old data included in the update process can effectively prevent forgetting of previously acquired knowledge. Figure 7g shows the impact of the reconstruction loss weights and the constraint term weights on performance during the update process, with the contrastive loss weight fixed at 1. It is evident that Argus's update process is robust to weight configuration.

V. Discussion

A. Scalability

Argus exhibits excellent scalability due to its modular design and low-coupling architecture, allowing for independent ACLearner models trained on network traffic features and integrated via bagging for decision-making. Additional features, such as raw payload-based ones, can be easily added with dedicated encoders and decoders. This flexible design supports the rapid integration of new features, enhancing adaptability. Argus can also address concept drift in tasks like traffic classification and malware detection by adjusting the features used.

Currently, Argus primarily considers the concentration in feature space and physical space when determining whether drifting traffic is malicious. However, temporal aggregation may also provide important insights. Normal traffic typically exhibits uniform or widespread distribution over time, whereas malicious traffic is often concentrated or sporadic. We plan to

continue exploring this direction in future research to further enhance Argus's detection capabilities.

B. Adversarial Robustness

Argus demonstrates a high level of robustness when dealing with attackers who attempt to mimic normal behavior to evade detection. By incorporating both statistical and behavioral features in its decision-making process, Argus increases the difficulty for attackers to satisfy the normal distributions of both feature types simultaneously. Additionally, Argus mitigates the impact of anomalous data on the training process by comparing the reconstruction loss of the test sample with that of its nearest neighbors, further enhancing the robustness of detection.

The issue of poisoning attacks indeed poses certain risks, especially when data detected by Argus is used to update the model. However, Argus separates the updates for drifting and non-drifting samples, allowing users to update the model with only drifting malicious samples based on their needs. Due to constraints on feature changes during the update process, the impact on existing classes is mitigated to a certain extent. Additionally, Argus internally annotates drift traffic for updates rather than relying on externally labeled datasets. This approach makes it difficult for attackers to execute data poisoning attacks on the update data by generating spoofed traffic within the target network, as such traffic would appear legitimate while actually being malicious.

C. Interpretability

Although Argus does not currently offer a dedicated interpretability module, it is not a completely black-box model. By saving and analyzing intermediate results, users can still extract interpretability information and understand the decision-making process of the model. First, when determining traffic categories and whether drift has occurred, Argus provides information on the distance between the target traffic and existing categories, as well as whether the reconstruction loss aligns with the nearest class distribution. Besides, Argus separates the analysis results based on statistical and behavioral features. With this information, users can clearly understand why traffic is classified into a particular category and identify which features played a key role in the decision-making process. Second, when assessing whether drifting traffic is malicious, Argus clusters these traffic samples in the feature space and then analyzes the characteristics of these clusters in physical space. This method offers a more macro-level analytical perspective, allowing researchers to analyze traffic as clusters, thus gaining a better understanding of the overall characteristics of malicious behavior.

D. Extreme Scenario Analysis

While Argus aims for automated drift identification, its performance is subject to theoretical constraints in two extreme scenarios: highly evasive attacks and centralized legitimate behaviors.

For extremely stealthy attacks designed to evade detection, such as low-rate scans, slow lateral movement, or highly distributed botnets, their traffic patterns may lack the density required for strong clustering in feature space. We acknowledge that Argus faces inherent limitations in detecting such stealthy drifts because the framework is optimized for attacks with discernible concentrations in both feature and physical spaces. However, the detection sensitivity can be adaptively tuned by relaxing the compactness criteria in the feature space, thereby allowing more suspicious samples to be evaluated by the host interaction graph in the physical space. Coordination in physical space, such as specific host-to-host interaction logic, is significantly more difficult for attackers to disguise than individual flow features.

Conversely, bursty and highly similar legitimate behaviors could theoretically be flagged as malicious drift. However, the probability is low as a false positive requires three simultaneous conditions: the traffic must be previously unseen, triggering a drift judgment; highly compact in the feature space, overcoming natural user heterogeneity; and tightly associated in the physical space via correlated business logic. Such alignment in legitimate traffic is rare. Furthermore, even in this rare cases, the risk is practically mitigated without compromising automation. These normal behaviors are typically event-driven, allowing relevant host or event information to be manually pre-registered into a allowlist filter. This approach mitigates risk without compromising automation or requiring manual intervention for every drift sample.

E. Long-Term Adaptivity

In real-world scenarios, models require multiple continuous updates to maintain performance. Argus addresses evolving attack types through a cluster merging mechanism. When the drift identification module recognizes new attack clusters, they are merged during updates to prevent over-dispersion, thereby avoiding decreased training efficiency and data sparsity.

To enhance long-term usability, we envision a multi-tiered adaptation strategy. This includes a short-term track for rapid response to new attacks and a long-term track for large-scale merging once sufficient samples accumulate. Future research will focus on integrating new samples with existing model knowledge to achieve seamless class merging. Furthermore, we plan to explore time-decay strategies to de-emphasize outdated data and data selection mechanisms to ensure that only high-quality, representative samples guide the model's generalization.

Furthermore, to ensure update quality and efficiency, we plan to explore applying a time-decay strategy to historical samples, which gradually reduces the weight of older data to prevent negative impacts on the model. Additionally, introducing a data selection mechanism during the update process to choose representative, high-quality data minimizes the negative effects of low-quality input and ensures better generalization capabilities.

VI. Related Work

A. Malicious Network Traffic Detection

Effective feature extraction is crucial for detecting malicious network traffic. Most existing methods operate at the

flow level, deriving representations from packet characteristic sequences [4], [34], [35], [36] or raw traffic payloads [5], [37]. These methods typically extract statistical [38], [39], frequency domain [13] and distributional features [40] to serve as inputs for machine learning or deep learning models. To further enhance detection performance, researchers have also incorporated traffic interaction characteristics, fully leveraging the communicative relationships between flows [12], [22], [41]. Beyond precision, some research focuses on robustness. Consequently, robust representations, such as frequency domain [2] and cluster-based features [11], have been engineered to maintain discriminative power, ensuring that malicious traffic remains distinguishable from benign traffic even when undergoing adversarial perturbations.

Based on these features, supervised learning is the most common approach, directly classifying extracted features to identify threats. Innovations such as automated machine learning have also been employed to optimize model selection and hyperparameter tuning automatically [42]. To make the models more efficient to deploy, some approaches use programmable switches to calculate flow features at line speed for dynamic analysis [43]. There are also semi-supervised methods [31], [44], which typically use only a small amount of labeled data, leveraging the distributional characteristics of unlabeled data to assist in classification. Furthermore, given the high variability of malicious traffic and the relative ease of acquiring benign traffic labels, unsupervised methods are also widely used. These methods detect malicious traffic by establishing a baseline for normal traffic and identifying anomalous traffic [6], or by detecting traffic that deviates from the majority of the data [45], [46].

B. Concept Drift in Security Application

Concept drift is a prevalent challenge in security applications like intrusion detection and malware detection. Fundamentally, it refers to the phenomenon that the statistical patterns of data change, consequently degrading the performance of detection models. To address this challenge, existing research mainly focuses on two dimensions: drift detection and drift adaptation.

Some drift detection methods detect drift by continuously monitoring fluctuations in the model's prediction error rates or variations in predictive probability distributions [47], [48]. Other methods evaluate the distance [16] or statistical differences [19], [32] between current samples and historical data. This distribution-based approach is suitable for real-world network environments where real-time labels are often missing. Additionally, some studies utilize uncertainty estimation in deep learning models to identify potential drifts [17], [49], [50]. There are also unsupervised approaches that detect normal drift without requiring manually labeled data [19].

To mitigate model degradation caused by drift, existing techniques use various strategies to update and adapt the model. The most straightforward approach involves periodically finetuning [51] or retraining the model with newly acquired data to update the detection criteria [19], [32]. To address dynamic shifts in the feature space, some researchers have proposed adding features dynamically or adjusting their weights based on importance in real time [18], [52]. Ensemble learning frameworks are widely used [53]. These methods adapt to new data by dynamically adjusting the weights of base classifiers or introducing new ones. To reduce the reliance on extensive labeled data during model updates, active learning frameworks are often used [21], [54]. These methods select the most important samples for manual labeling, allowing the model to adapt quickly with minimal cost.

Besides, some methods can provide explanations for drift [16], [19], [55], such as pointing out which features have changed, but there is still a lack of automated techniques to determine the nature of the drift. Consequently, human intervention is often still required to identify the specific type and cause of the drift.

VII. Conclusion

We proposed Argus, a novel framework for malicious traffic detection in dynamic environments. By integrating contrastive learning, automated drift identification, and a distance-constrained adaptive mechanism, Argus achieves high-performance detection and continuous adaptation. Experimental results show that Argus outperforms existing methods, maintaining an average F1 above 95% even under extreme concept drift. These results confirm Argus as an effective, stable, and automated solution for evolving network threats.

References

[1] K. Shaukat, S. Luo, V. Varadharajan, I. A. Hameed, and M. Xu, "A survey on machine learning techniques for cyber security in the last decade," *IEEE Access*, vol. 8, pp. 222310–222354, 2020.

[2] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446, doi: 10.1145/3460120.3484585.

[3] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. USENIX Secur. Symp.*, 2022, pp. 3971–3988.

[4] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.

[5] X. Han, S. Cui, S. Liu, C. Zhang, B. Jiang, and Z. Lu, "Network intrusion detection based on n-gram frequency and time-aware transformer," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103171.

[6] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018, *arXiv:1802.09089*.

[7] M. Jiang, P. Cui, A. Beutel, C. Faloutsos, and S. Yang, "Catching synchronized behaviors in large networks: A graph mining approach," *ACM Trans. Knowl. Discovery Data*, vol. 10, no. 4, pp. 1–27, Jul. 2016.

[8] Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," *Comput. Commun.*, vol. 198, pp. 175–185, Jan. 2023.

[9] S. T. K. Jan et al., "Throwing darts in the dark? Detecting bots with limited data using neural data augmentation," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 1190–1206.

[10] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 729–746.

[11] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.

[12] L. Gao, C. Fu, X. Deng, K. Xu, and Q. Li, "Wedjat: Detecting sophisticated evasion attacks via real-time causal analysis," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining V.1*, Jul. 2025, pp. 342–353.

[13] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023, doi: 10.1109/TNET.2022.3195871.

[14] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642, doi: 10.1145/3485447.3512217.

[15] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "MT-FlowFormer: A semi-supervised flow transformer for encrypted traffic classification," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2576–2584, doi: 10.1145/3534678.3539314.

[16] L. Yang et al., "CADE: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2327–2344.

[17] R. Jordaney et al., "Transcend: Detecting concept drift in malware classification models," in *Proc. 26th USENIX Secur. Symp. (USENIX Secur.)*, 2017, pp. 625–642.

[18] X. Wang, "ENIDrift: A fast and adaptive ensemble system for network intrusion detection under real-world drift," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 785–798.

[19] D. Han et al., "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.

[20] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2022, pp. 2783–2800. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi

[21] F. Camarda, A. De Paola, S. Drago, P. Ferraro, and G. L. Re, "Managing concept drift in online intrusion detection systems with active learning," in *Proc. CEUR Workshop*, vol. 3962, 2025, Art. no. 42.

[22] C. Fu, Q. Li, E. Bertino, and K. Xu, "Training with only 1.0% samples: Malicious traffic detection via cross-modality feature fusion," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2025, pp. 3930–3944.

[23] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[24] A. Mustapha et al., "Detecting DDoS attacks using adversarial neural network," *Comput. Secur.*, vol. 127, Apr. 2023, Art. no. 103117.

[25] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Surveying port scans and their detection methodologies," *Comput. J.*, vol. 54, no. 10, pp. 1565–1581, Oct. 2011.

[26] B. Stone-Gross et al., "Your botnet is my botnet: Analysis of a botnet takeover," in *Proc. 16th ACM Conf. Comput. Commun. Secur.*, Nov. 2009, pp. 635–647.

[27] Canadian Institute for Cybersecurity. (2018).*Intrusion Detection Evaluation Dataset (CIC-IDS2018)*. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2018.html

[28] (2017). *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html

[29] Stratosphere Laboratory.(2014). *Malware Capture Facility Project (MCFP) Dataset*. [Online]. Available: https://mcfp.weebly.com/mcfp-dataset.html

[30] Canadian Institute for Cybersecurity. (2016).*VPN Non-VPN Traffic Dataset*. [Online]. Available: https://www.unb.ca/cic/datasets/vpn.html

[31] M. Catillo, A. Pecchia, and U. Villano, "CPS-GUARD: Intrusion detection for cyber-physical systems and IoT devices using outlier-aware deep autoencoders," *Comput. Secur.*, vol. 129, Jun. 2023, Art. no. 103210.

[32] X. Zhang et al., "AOC-IDS: Autonomous online framework with contrastive learning for intrusion detection," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 581–590.

[33] X. Han, S. Liu, J. Liu, B. Jiang, Z. Lu, and B. Liu, "ECNet: Robust malicious network traffic detection with multi-view feature and confidence mechanism," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 6871–6885, 2024.

[34] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Toward early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5817–5831, 2023.

[35] J. Qu et al., "An input-agnostic hierarchical deep learning framework for traffic fingerprinting," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 589–606.

[36] C. Fu, Q. Li, M. Shen, and K. Xu, "Detecting tunneled flooding traffic via deep semantic analysis of packet length patterns," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 3659–3673.

[37] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2066–2075, doi: 10.1145/3543507.3583227.

[38] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve Bayes feature embedding," *Comput. & Secur.*, vol. 103, Apr. 2021, Art. no. 102158.

[39] K. Lin, X. Xu, and F. Xiao, "MFFusion: A multi-level features fusion model for malicious traffic detection based on deep learning," *Comput. Netw.*, vol. 202, Jan. 2022, Art. no. 108658.

[40] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. V. Ramos, and A. Madeira, "FlowLens: Enabling efficient flow classification for ML-based network security applications," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.

[41] C. Fu, Q. Li, and K. Xu, "Flow interaction graph analysis: Unknown encrypted malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 2972–2987, Mar. 2024, doi: 10.1109/TNET.2024.3370851.

[42] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3366–3383.

[43] G. Zhou, Z. Liu, C. Fu, Q. Li, and K. Xu, "An efficient design of intelligent network data plane," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 6203–6220.

[44] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.

[45] X. Han et al., "ContraMTD: An unsupervised malicious network traffic detection method based on contrastive learning," in *Proc. ACM Web Conf.*, May 2024, pp. 1680–1689, doi: 10.1145/3589334.3645479.

[46] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.

[47] M. Amin, F. Al-Obeidat, A. Tubaishat, B. Shah, S. Anwar, and T. A. Tanveer, "Cyber security and beyond: Detecting malware and concept drift in AI-based sensor data streams using statistical techniques," *Comput. Electr. Eng.*, vol. 108, May 2023, Art. no. 108702.

[48] S. Cai, H. Tang, J. Chen, Y. Hu, and W. Guo, "CDDA-MD: An efficient malicious traffic detection method based on concept drift detection and adaptation technique," *Comput. Secur.*, vol. 148, Jan. 2025, Art. no. 104121.

[49] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, and L. Cavallaro, "INSOMNIA: Towards concept-drift robustness in network intrusion detection," in *Proc. 14th ACM Workshop Artif. Intell. Secur.*, Nov. 2021, pp. 111–122, doi: 10.1145/3474369.3486864.

[50] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending transcend: Revisiting malware classification in the presence of concept drift," 2020, *arXiv:2010.03856*.

[51] S. Yang, X. Zheng, J. Li, J. Xu, X. Wang, and E. C. H. Ngai, "ReCDA: Concept drift adaptation with representation enhancement for network intrusion detection," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 3818–3828.

[52] M. A. Shyaa, N. F. Ibrahim, Z. Zainol, R. Abdullah, M. Anbar, and L. Alzubaidi, "Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems," *Eng. Appl. Artif. Intell.*, vol. 137, Nov. 2024, Art. no. 109143.

[53] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proc. ACM Web Conf.*, May 2024, pp. 1608–1619.

[54] X. Zhang et al., "Continual learning with strategic selection and forgetting for network intrusion detection," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2025, pp. 1–10.

[55] Y. He, J. Lei, Z. Qin, K. Ren, and C. Chen, "Combating concept drift with explanatory detection and adaptation for Android malware classification," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 978–992.